

SILworX Plug-In O_Scope

```
#----- OPERATOR CLASSES -----  
# Mirror Tool  
class MirrorX(bpy.types.Operator):  
    """This adds an X mirror to the selected object"""  
    bl_idname = "object.mirror_mirror_x"  
    bl_label = "Mirror X"  
  
    @classmethod  
    def poll(cls, context):  
        return context.active_object is not None  
        mirror_mod = modifier_ob.modifiers["Mirror X"]  
  
    # set mirror object to mirror_ob  
    mirror_mod.mirror_object = mirror_ob  
  
    if _operation == "MIRROR X":  
        mirror_mod.use_x = True  
        mirror_mod.use_y = False  
        mirror_mod.use_z = False  
    elif _operation == "MIRROR Y":  
        mirror_mod.use_x = False  
        mirror_mod.use_y = True  
        mirror_mod.use_z = False  
    elif _operation == "MIRROR Z":  
        mirror_mod.use_x = False  
        mirror_mod.use_y = False  
        mirror_mod.use_z = True  
  
    #selection at the end -add back the modifier  
    mirror_ob.select= 1  
    modifier_ob.select= 1  
    bpy.context.select= 1  
    print("Selected" + str(modifier_ob))  
    #mirror_ob.select = 0  
    #bpy.data.objects[selected_objects[0]].select = 1  
except:  
    print("please select exactly two objects")  
  
#----- OPERATOR CLASSES -----  
# Mirror Tool  
class MirrorX(bpy.types.Operator):  
    """This adds an X mirror to the selected object"""  
    bl_idname = "object.mirror_mirror_x"  
    bl_label = "Mirror X"  
  
    @classmethod  
    def poll(cls, context):  
        return context.active_object is not None  
        mirror_mod = modifier_ob.modifiers["Mirror X"]  
  
    # set mirror object to mirror_ob  
    mirror_mod.mirror_object = mirror_ob  
  
    if _operation == "MIRROR X":  
        mirror_mod.use_x = True  
        mirror_mod.use_y = False  
        mirror_mod.use_z = False  
    elif _operation == "MIRROR Y":  
        mirror_mod.use_x = False  
        mirror_mod.use_y = True  
        mirror_mod.use_z = False  
    elif _operation == "MIRROR Z":  
        mirror_mod.use_x = False  
        mirror_mod.use_y = False  
        mirror_mod.use_z = True  
  
    #selection at the end -add back the modifier  
    mirror_ob.select= 1  
    modifier_ob.select= 1  
    bpy.context.select= 1  
    print("Selected" + str(modifier_ob))  
    #mirror_ob.select = 0  
    #bpy.data.objects[selected_objects[0]].select = 1  
except:  
    print("please select exactly two objects")  
  
#----- OPERATOR CLASSES -----  
# Mirror Tool  
class MirrorX(bpy.types.Operator):  
    """This adds an X mirror to the selected object"""  
    bl_idname = "object.mirror_mirror_x"  
    bl_label = "Mirror X"  
  
    @classmethod  
    def poll(cls, context):  
        return context.active_object is not None  
        mirror_mod = modifier_ob.modifiers["Mirror X"]  
  
    # set mirror object to mirror_ob  
    mirror_mod.mirror_object = mirror_ob  
  
    if _operation == "MIRROR X":  
        mirror_mod.use_x = True  
        mirror_mod.use_y = False  
        mirror_mod.use_z = False  
    elif _operation == "MIRROR Y":  
        mirror_mod.use_x = False  
        mirror_mod.use_y = True  
        mirror_mod.use_z = False  
    elif _operation == "MIRROR Z":  
        mirror_mod.use_x = False  
        mirror_mod.use_y = False  
        mirror_mod.use_z = True  
  
    #selection at the end -add back the modifier  
    mirror_ob.select= 1  
    modifier_ob.select= 1  
    bpy.context.select= 1  
    print("Selected" + str(modifier_ob))  
    #mirror_ob.select = 0  
    #bpy.data.objects[selected_objects[0]].select = 1  
except:  
    print("please select exactly two objects")
```

All of the HIMA products mentioned in this manual are trademark protected. Unless noted otherwise, this also applies to other manufacturers and their respective products referred herein.

HIQuad®, HIQuad® X, HIMax®, HIMatrix®, SILworX®, XMR® and HICore® are registered trademarks of HIMA Paul Hildebrandt GmbH.

All of the technical specifications and information in this manual were prepared with great care and effective control measures were employed for their compilation. For questions, please contact HIMA directly. HIMA appreciates any suggestion on which information should be included in the manual.

Equipment subject to change without notice. HIMA also reserves the right to modify the written material without prior notice.

Registered customers can download the product documentation from the HIMA Extranet.

© Copyright 2024, HIMA Paul Hildebrandt GmbH

All rights reserved.

Contact

HIMA Paul Hildebrandt GmbH

P.O. Box 1261

68777 Brühl, Germany

Tel.: +49 6202 709-0

Fax: +49 6202 709-107

E-Mail: info@hima.com

Document designation	Description
SILworX Plugin O_Scope D Rev. 1.2 (2428)	German original document
SILworX Plugin O_Scope E Rev. 1.1 (2428)	English translation of the German original document

Table of Contents

1	SILworX Plug-Ins (Option Subject to License)	5
2	Configuring Plug-Ins	6
2.1	Installing Plug-Ins Using the Plug-In Manager	6
2.2	Manual Configuration of the Plug-In	6
2.2.1	Creating the SSL Certificate and Key	7
2.2.2	Completing the «settings.ini»	7
2.2.3	Plug-In-Registration	8
2.2.4	Starting Plug-Ins	9
2.2.5	Completing the «plugins.json»	9
3	Functionality of the Plug-In «O_scope»	11
3.1	Buttons and Input Fields	14
3.2	Shortcuts and Key Combinations	16

1 SILworX Plug-Ins (Option Subject to License)

SILworX can be equipped with plug-ins to enhance its functionality. Plug-ins are non-safety-related applications that interact with SILworX, but are not contained in the SILworX development environment and are not certified along with SILworX.

Plug-ins enable users to create additional functions according to their own needs or to make these available to others. The prerequisite is that the necessary licenses exist. Plug-ins created by the customer can be used with the Plug-In Feature license in combination with the SILworX API license. Use of the license-free plug-ins provided by HIMA requires only the SILworX license.

Plug-ins run as independent processes in a memory area separate from SILworX and interact with SILworX via the *SILworX API* and *WebSocket* interfaces using encrypted connections. Plug-ins can be integrated interference-free and do not impair the safe SILworX functions in any way.

Because plug-ins are not an integral component of SILworX, they must be configured in the *settings.ini* file. To this end, the *settings.ini* file includes the *[Plugin_Server]* category. When started, SILworX checks whether the registered plug-ins exist and readies them so the functionality they offer is available in SILworX. SILworX starts the plug-ins listed in *settings.ini* under *PluginNames* automatically. The sequence of the plug-in names determines the processing priority of the plug-ins in the event of simultaneous triggering.

Plug-ins evaluate predefined triggers provided by SILworX as well as user-specific triggers and execute the functions required by the user. The actions triggered in SILworX by plug-ins are logged in the logbook. A separate area to the right of the API information in the Status Bar displays information about the plug-in feature.

The plug-ins that are started have no separate user management. Authentication takes place when the project is opened in SILworX. The user session this creates is also available to the plug-ins and the SILworX API.

When the plug-in feature has been started, you can use the **Plug-In Information** menu function in the SILworX **Help** menu to open a dialog box. The dialog box contains the two lists *Configured Plug-Ins* and *Registered Plug-Ins*. The dialog box shows the actions with which a plug-in in SILworX is linked and which triggers from SILworX or to SILworX are registered.

Summary:

- Plug-ins are SILworX add-ons that can be created by users themselves.
- Plug-ins are based on programs or scripts, possibly with additional files, that are configured in SILworX.
- Plug-ins are independent modules that run in separate processes.
- Plug-ins are started automatically along with SILworX.
- Plug-ins have access to SILworX via the SILworX API functions.
- Plug-ins require a Plug-In Feature license as well as a SILworX API license.
- Plug-ins can react to predefined triggers as well as user-defined triggers.
- The execution of plug-ins is displayed in the SILworX status bar.

2 Configuring Plug-Ins

If you wish to enhance SILworX with additional functions using plug-ins, you must configure the required plug-ins correctly in SILworX. The processes this entails require numerous steps.

You can either carry out these steps completely manually or let *Plug-In Manager* assist you. The *Plug-In Manager* itself is a license-free plug-in provided by HIMA that you may use free of charge. If you select the options *Create a TLS Certificate for SILworX API and Plug Ins* and *Activate HIMA Plug Ins* when installing SILworX, then the additional menu functions **Install Plug Ins** and **Uninstall Plug Ins** are available in the SILworX **Extras** menu

2.1 Installing Plug-Ins Using the Plug-In Manager

To install the plug-in or to update an existing plug-in, open the **Installing Plug-Ins** menu item in the **Extras** menu. A dialog box will prompt you to select a plug-in archive with the extension «.silworx_plugin». The metadata from the plug-in archive are displayed to identify the selected plug-in. When the **Install** button is confirmed, the plug-in is added to SILworX.

If SILworX already contains a plug-in whose name is identical with name from the selected plug-in archive, the meta information with the differences is displayed. When the **Update** button is clicked, the existing plug-in is replaced by the new plug-in. The replacement takes place even if there are differences in the meta information.

2.2 Manual Configuration of the Plug-In

To use SILworX plug-ins requires that you save the desired plug-ins to a drive that is accessible to SILworX and configure them in the *settings.ini* file. You also require a Plug-In Feature license, a SILworX API license and a SILworX API certificate. Access to the SILworX API is only possible via SSL (TLS 1.2 or TLS 1.3).

The use of SSL requires a valid certificate and the associated private key. The paths to these files must be entered in the *settings.ini* file in the category *[API_Server]*. Using plug-ins in combination with the SILworX API requires that the configuration parameter *Enabled* of the category *[API_Server]* in the *settings.ini* is set to TRUE.

When starting, SILworX imports various plug-in configuration parameters from the *settings.ini* file, which are grouped in the category *[Plugin_Server]*. After this, SILworX checks the integrity of the registered plug-ins based on the hash codes. If the result of the integrity check is positive, SILworX makes the plug-in functions available.

The names of the plug-ins to be used in SILworX must be present in the *PlugInNames* list separated by commas. If several plug-ins respond to a certain trigger, their sequence in the *PlugInNames* list determines the order in which the plug-ins are executed.



A maximum of 254 plug-ins can be registered.

If *PlugInNames* contains plug-in names that occur more than once, the duplicates are rejected during the import of the plug-in configuration and SILworX reports an error.



The plug-in names are defined in the metadata of the plug-ins. They can differ from the names of the files used for the plug-ins.

The plug-ins listed in *PlugInNames* must be executable programs (*.exe) and, to protect against unauthorized tampering, each of these plug-ins must be configured in the hash file *plugins.json*.



SILworX installs several HIMA plug-ins and enters their names as *PluginNames* by default. You can supplement this list at any location with your own *PluginNames*. The API server must also be [Enabled](#) to allow use of the HIMA plug-ins.

After initialization, SILworX automatically starts all of the plug-ins configured in *PluginNames*. Doing so requires that the appropriate hash codes for the plug-ins listed in *PluginNames* are present in *plugins.json*. The data paths are also taken from *plugins.json*. SILworX issues messages to the logbook if plug-ins cannot be started.

2.2.1 Creating the SSL Certificate and Key

To use the SILworX API, you need an SSL certificate in the PEM format, which you must create yourself. In addition to the certificate, you also need the corresponding private key in the PEM format that was created with the RSA algorithm without a passphrase.

The certificate and key must be entered in the SILworX *settings.ini* file.



Observe the period of validity of the SSL certificate!

When you create a new SSL certificate, you can define its period of validity in days. HIMA recommends limiting the period of validity to 365 days or less. Some clients (e.g., current browsers) do not accept any self-signed certificates with a validity period of greater than 365 days.

To create a valid certificate

- Select in SILworX **Extras, Create SILworX-API certificate** from the main menu. The *Create SILworX-API certificate* dialog box appears.
- Enter a path for the destination directory and the certificate name in the *Filepath - Certificate* field or use the button on the right-hand side, next to the field, to specify the path and name in a dialog box.
- Enter a path for the destination directory and the key name in the *Filepath - Key* field or use the button on the right-hand side, next to the field, to specify the path and name in a dialog box.
- If required, fill in the optional fields with your data.
- Click **Create** to confirm the action.

2.2.2 Completing the «settings.ini»

To configure the SILworX API and the plug-ins, you must complete the following entries to the *settings.ini* file.

- Open the *settings.ini* file in the SILworX program data directory %ProgramData%\SILworX_vXX.X.0 Rxxxx\settings\, for example C:\ProgramData\SILworX_v15.0.0 R2875\settings\settings.ini, with a text editor.
- Search in the *settings.ini* file for the entry *[API_Server]*. If this entry does not exist, append it to the end of the file.
- The following entries are required under *[API_Server]*, and you must complete them as needed.

[Enabled=true](#)

[ServerCertificate=Absolute path to the certificate file](#)

[ServerPrivateKey=Absolute path to the key file](#)

- Search in the *settings.ini* file for the entry *[Plugin_Server]*. If this entry does not exist, append it to the end of the file.
- The following entries are required under *[Plugin_Server]*, and you must complete them as needed.

Enabled=true

PluginNames=List of plug-ins names separated by commas

Example for the «settings.ini»

```
...  
[API_Server]  
Enabled=true  
ServerCertificate=C:\Plugins\Certificates\api_cert.pem  
ServerPrivateKey=C:\Plugins\Certificates\api_key.pem  
[Plugin_Server]  
Enabled=true  
PluginNames=hima.o_scope
```

2.2.3 Plug-In-Registration

In the registration that follows a successful WebSocket connection set-up, the plug-ins log in to SILworX with their names, the metadata and their required triggers. This requires that the plug-ins be correctly configured with their names in *settings.ini*.

Plug-ins log in after they are started by means of messages, which they send to SILworX. If the registration was successful, messages with the file path of the plug-ins are written to the logbook. Also the hash codes of the plug-ins are shown.

Avoiding Errors During Plug-In Registration

If the registration was not successful, an error message is displayed in the logbook with an indication of the cause of the failed registration. To avoid errors, observe the following notes:

- **Multiple registration of plug-ins**
No duplicate names may occur in the configuration parameter *PluginNames* of *settings.ini*.
- **Inadmissible registration of a plug-in**
The plugin name defined in the metadata of a plug-in must be listed in the configuration parameter *PluginNames* of *settings.ini*. This plugin name must also be assigned as a value to the *plugin_name* key in *plugins.json*.
- **Maximum number of plug-ins**
In SILworX, you can register a maximum of 254 plug-ins simultaneously.
- **Maximum number of triggers**
The maximum number of 20 trigger entries for the **Extras** menu and 100 trigger entries for the total of all context menus must not be exceeded.

- **Incorrect formatting of hash file**

Pay attention to the correct number of opening and closing brackets and double quotation marks. Whitespaces outside of string values are not significant. The path to the plug-in file must contain slashes / or double backslashes \\, single backslashes \ are not permitted.

2.2.4 Starting Plug-Ins

Starting plug-ins means that plug-ins are first registered in SILworX so they can interact with SILworX. After successful plug-in registration, the programs are present in the working memory of the computer. The actual interactions with SILworX and data processing are triggered by trigger notifications that SILworX sends to the plug-ins.

Upon initialization, SILworX starts a plug-in process for every plug-in that is configured in *settings.ini* under *PluginNames*, provided the plug-in data are entered in the plug-in hash file. If these data do not exist, SILworX issues a message and does not start the plug-in.

SILworX starts the plug-ins listed in the *settings.ini* automatically. When a plug-in is started, SILworX passes a command line with some parameters and generates a message with the content of the command line in the logbook.

The plug-ins that are started have no separate user management. Authentication takes place when the project is opened in SILworX. The user session this creates is also available to the plug-ins and the SILworX API.

2.2.5 Completing the «plugins.json»

The authenticity of the plug-ins configured in *settings.ini* under *PluginNames* is checked by SILworX before they are registered. The check is made by comparing the currently calculated hash codes of the plug-ins with the nominal values provided by the producer of the plug-ins that were previously stored in a hash file.

When importing the plug-in configuration and hash file, SILworX discards those plug-in names that occur twice and reports them as errors.

The hash file *plugins.json* contains the hash file codes of the plug-ins and must be adapted by the customer as needed. The file contains a separate data block for each plug-in and is structured as follows:

```
{
  "plugins": [
    {
      "plugin_hash": "Hash code according to SHA-256 plug-in 1",
      "plugin_name": "Plug-in name according to the meta data of plug-in 1",
      "plugin_path": "Absolute path to plug-in 1"
    },
    {
      "plugin_hash": "Hash code according to SHA-256 plug-in 2",
      "plugin_name": "Plug-in name according to the meta data of plug-in 2",
      "plugin_path": "Absolute path to plug-in 2"
    }
  ]
}
```

To configure the plug-ins, you must complete the following entries to the *plugins.json* file.

- Open the *plugins.json* file in the SILworX program data directory
%ProgramData%\SILworX_vXX.X.0 Rxxxx\plugins\, for example
C:\ProgramData\SILworX_v15.0.0 R2875\plugins\customplugins.json, with a text editor.
- The hash file contains a separate data block for each plug-in, consisting of a list of key : value pairs enclosed in curly brackets:

```
{  
    "plugin_hash" : "Hash code according to SHA-256",  
    "plugin_name" : "Plug-in name according to the meta data of the plug-in",  
    "plugin_path" : "Absolute path to the plug-in"  
}
```

Add the appropriate values for the plug-in to this data block in the *plugins.json* file. If this data block does not exist, insert it after the opening square bracket. Please note that the string values must be enclosed in double straight quotation marks ".



Backslashes \ in JSON strings

In accordance with the JSON format definition, the backslash \ is used in JSON strings as a masking character to introduce escape sequences. Therefore, the backslashes in path specifications must either be replaced by double backslashes \\ or by slashes /.

Example for the «plugins.json»

```
{  
  "plugins": [  
    {  
      "plugin_hash":  
        "5BC0241544FF642A5B47BF3365391765386E46B8E3BEC347DA4D0D1E16698496",  
      "plugin_name": "hima.o_scope",  
      "plugin_path": "C:/Plugins/O_Scope.exe"  
    }  
  ]  
}
```

3 Functionality of the Plug-In «O_scope»

The plug-in is started via the context menu of a program:

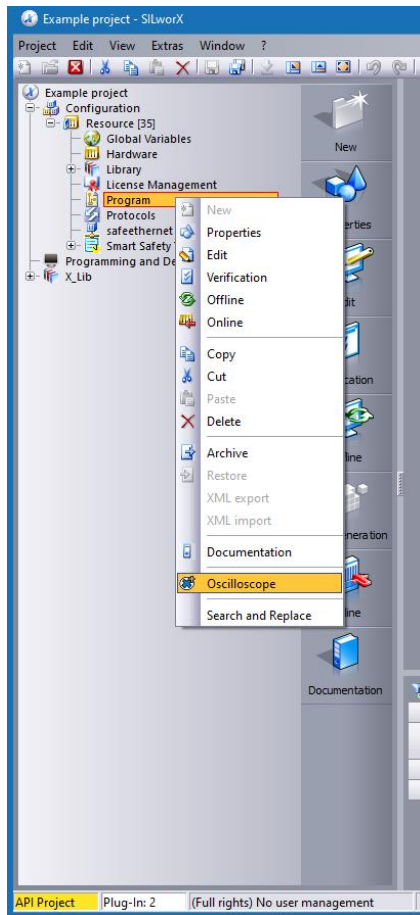


Figure 1: Program context menu with menu entry for the plug-in

The plug-in performs a system login to the running controller and reads all visualizable local variables of the user program selected in the structure tree. The variables are listed hierarchically in a structure tree. For variables selected by the user, their current process values are read cyclically from the controller and displayed graphically as a function of time.

To do this, the plug-in executes the following SILworX actions one after the other:

- Retrieving of the project name, configuration name, resource name and program name from the structure tree
- Execution of a system login via the **Extras, Online** menu of the higher-level resource
- Retrieving of the average cycle time of the connected resource from the control panel
- Retrieving of all local variables of the selected program via the **Forcing, Force Editor, Local Variables** menu
- Disconnecting the online session of the connected resource via the **Online, Disconnect** menu

After selecting the desired length of time for the chart and the variables to be displayed, the following actions are carried out after pressing the **Start** button:

- Execution of a system login via the **Extras, Online** menu of the selected program
- Cyclical reading of the process values of the selected program-local variables until the **Stop** button or is pressed or the plug-in window is closed.
- Disconnecting the online session of the connected resource via the **Online, Disconnect** menu

After start, the plug-in requests the login data for the controller via a dialog.

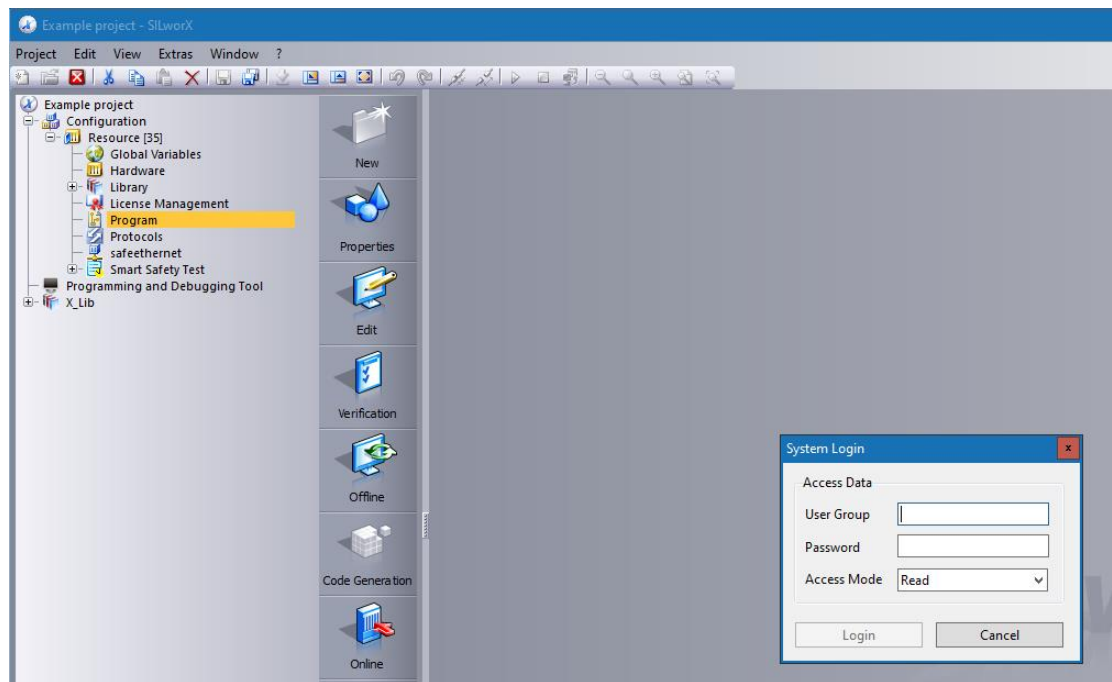


Figure 2: Login dialog of the plug-in for the system login

After entering the access data and pressing the **Login** button, the plug-in displays a message about retrieving the required data from the controller.

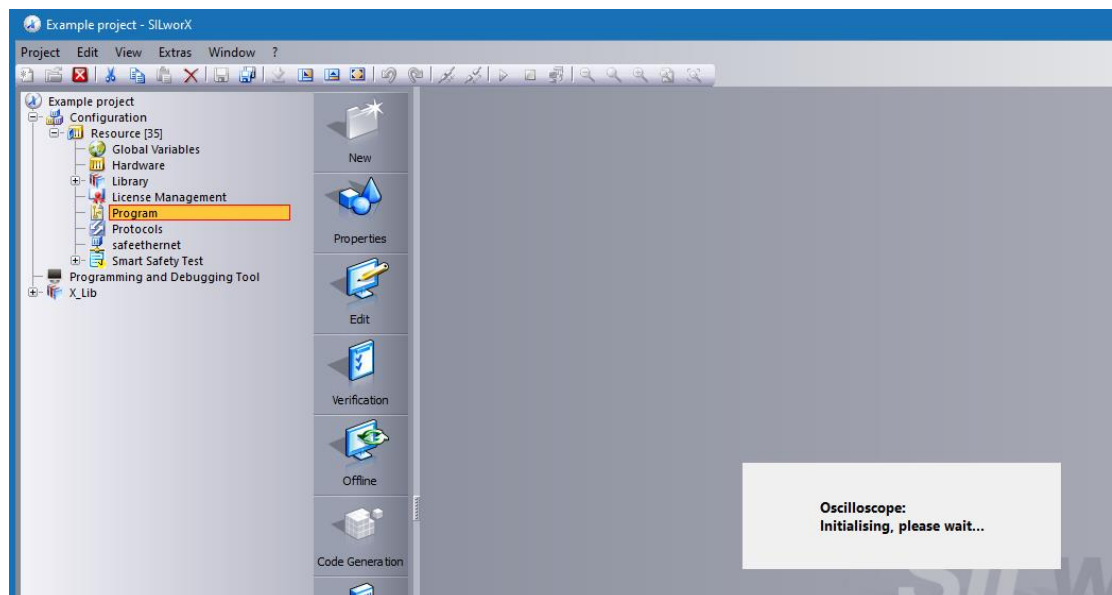


Figure 3: Initializing the plug-in

Once all the data has been retrieved, the main window of the plug-in appears.

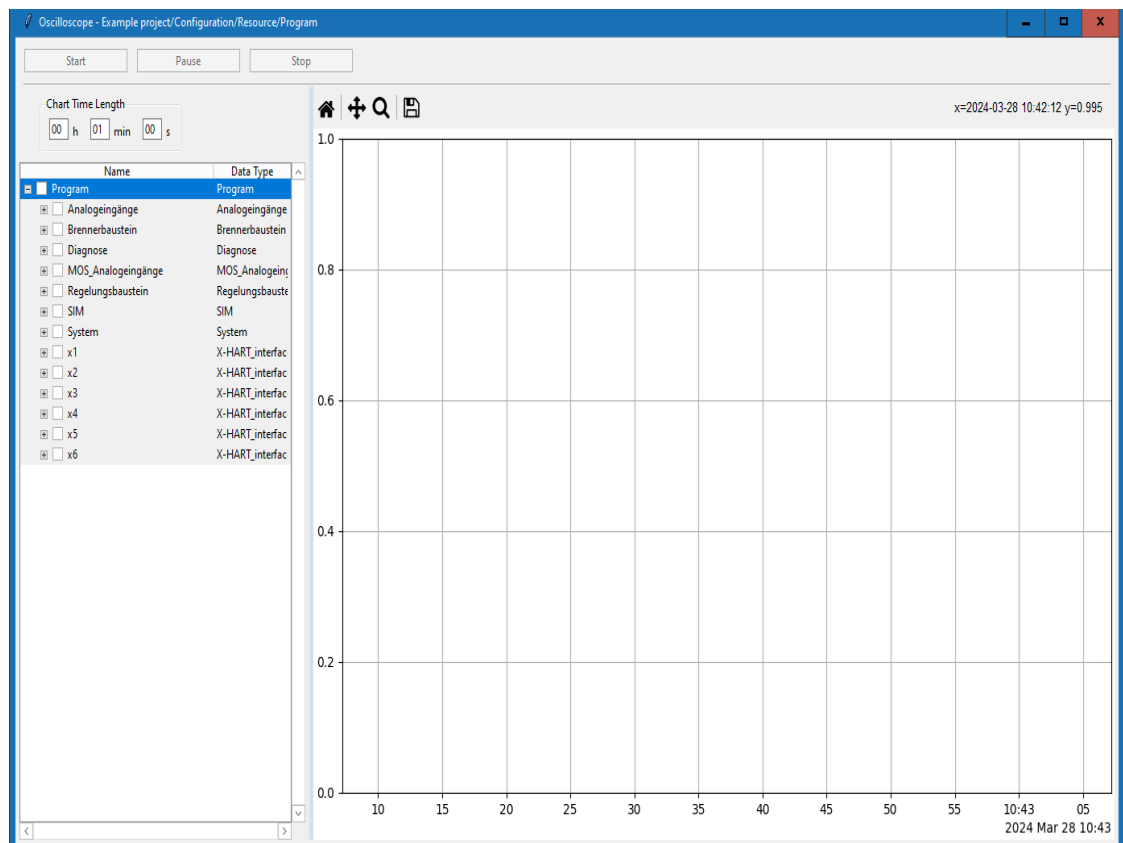


Figure 4: Main window of the plug-in with variable selection tree

You can select up to ten variables from the structure tree whose progression over time is to be displayed. The selection is made by clicking in the checkbox in front of the variable name or by double-clicking in the corresponding line. The length of the visible graph is defined via the **Chart Time Length** input. After selecting the variables to be displayed and specifying the duration, the recording of the process values is started by clicking the **Start** button.



Variables of data type *TIME* are interpreted as second values, i.e. 1000 ms in SILworX correspond to a value of 1.0 in the trend.

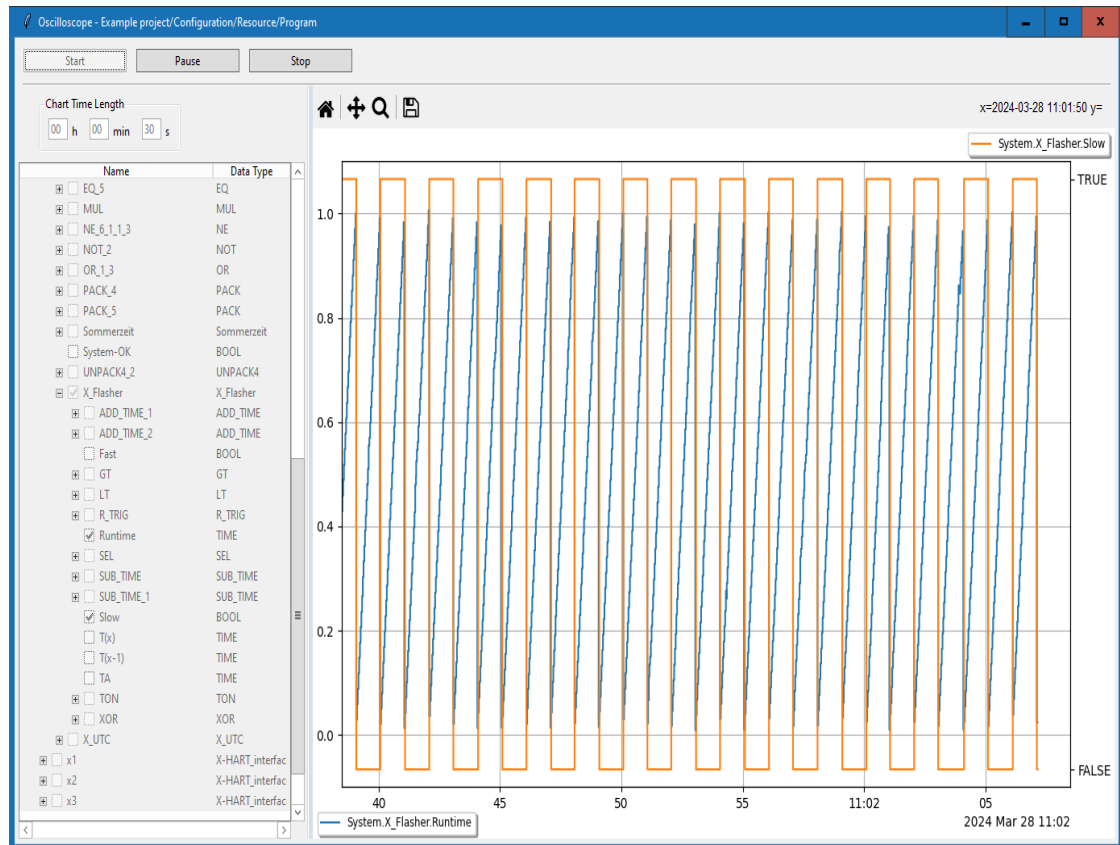


Figure 5: Graphic visualization of process values as a function of time

The process values read out cyclically during runtime are temporarily stored by SILworX in two binary files in the %Temp%\SILworX_Plugins directory. The process values are queried with twice the average cycle time of the controller, but at 30 ms at the fastest. If the duration of the diagram to be displayed is long, the query cycle may be increased to limit the amount of data to be handled.

The plug-in creates a log file with the name «hima.o_scope.log» in the %AppData%\SILworX_Plugins\ directory. The log file contains all status and error messages that have occurred and the time at which the message was issued. The output is in tabular form.

3.1 Buttons and Input Fields

Start, Pause, Resume, Stop

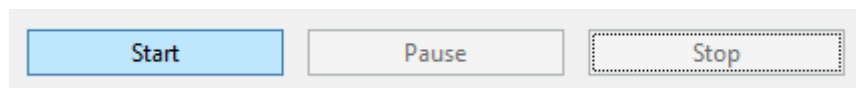
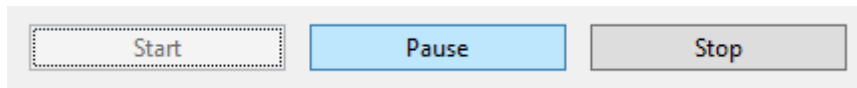
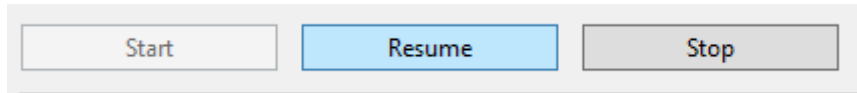
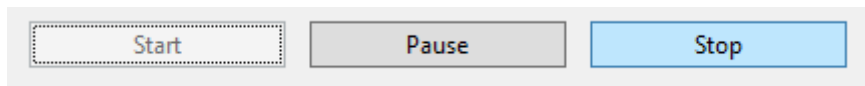


Figure 6: Button **Start**

The **Start** button is enabled as soon as a valid value has been entered for the Chart Time Length and at least one variable has been selected for visualization. After starting the recording, it is no longer possible to change the Chart Time Length or the variable selection in the structure tree until the recording is stopped.

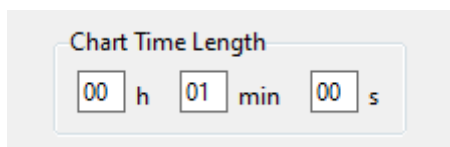
Figure 7: Button **Pause**Figure 8: Button **Resume**

The **Pause** button is available after starting the recording. When pressed, the current diagram is no longer updated to facilitate evaluation. The query of the process values from the controller continues to run in the background. It is not possible to change the configured recording parameters. Updating the trend curve is continued at the current point after pressing the **Resume** button.

Figure 9: Button **Stop**

The **Stop** button is enabled immediately after starting the recording. After the recording is stopped, the query of the process values from the controller is also stopped and the online session with the resource is disconnected. The Chart Time Length input and the structure tree for variable selection accept user input again.

Chart Time Length

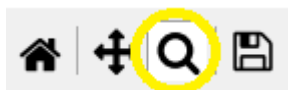
Figure 10: Input fields **Chart Time Length**

These input fields determine the duration of the visible trend curve. The smallest permissible value is ten seconds, the largest permissible value is five hours, 59 minutes and 59 seconds. The Start button can only be activated once a valid time period has been entered.

Buttons in the Trend Window

Figure 11: Button **Save Picture**

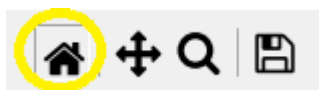
The **Save Picture** button opens a dialog to save the current image in various graphic formats.

Figure 12: Button **Zoom to Rectangle**

Pressing the **Zoom to Rectangle** button switches the zoom function on or off. When this function is activated, you can zoom in and out of the trend graph.

Figure 13: Button **Pan**

Pressing the **Pan** button switches the pan function on or off. When this function is activated, the trend graph can be moved on the X and Y axes without zooming.

Figure 14: Button **Home**

Clicking the **Home** button resets the original view after zooming or panning.

3.2 Shortcuts and Key Combinations

The usual SILworX shortcuts and key combinations are available:

System Login

Shortcut, key combination	Description
Ctrl+A	Sets the user name, password and access mode to the default factory setting Administrator

Input Fields Chart Time Length

Shortcut, key combination	Description
Tab	Moves the selection one cell to the right
Shift+Tab	Moves the selection one cell to the left

Structure Tree

Shortcut, key combination	Description
↑ ↓ ← →	Moves the selection up, down, left or right by one (visible) element
+, -, Enter key	Expands or collapses the structure tree
Space bar	Expands or collapses the structure tree Enables or disables the checkbox

Ctrl+Home	Moves the selection to the beginning of the structure tree
Ctrl+End	Moves the selection to the end of the structure tree
Character key	Moves the selection to the next element on the same hierarchy level that begins with this letter

Trend

Shortcut, key combination	Description
H, R, Home	Resets original view
P	Switches the pan function on or off
O	Switches the zoom-to-rect function on or off
S, Ctrl+S	Opens a dialog to save the current image in various graphic formats
LEFT MOUSE BUTTON	If pan function is switched on: • Move axes in any direction If zoom function is switched on: • Zoom in from a drawn rectangle
RIGHT MOUSE BUTTON	If pan function is switched on: • Increase or decrease axis scaling If zoom function is switched on: • Zoom out of a drawn rectangle
X+pan/zoom	Constrain pan/zoom to x axis
Y+pan/zoom	Constrain pan/zoom to y axis

Appendix

Glossary

Term	Description
SILworX	Programming tool for HIMax, HIQuad X and HIMatrix
API	Application Programming Interface
HTTPS	HyperText Transfer Protocol Secure, communication protocol
SSL	Secure Sockets Layer, deprecated ancestor of TLS
TLS	Transport Layer Security, cryptographic protocol
PEM	Privacy Enhanced Mail, file format according RFC 7468
RSA	Rivest–Shamir–Adleman, public-key cryptosystem
WebSocket	TCP based communications protocol according RFC 6455
SHA	Secure Hash Algorithm, cryptographic hash function
JSON	JavaScript Object Notation, file format according RFC 8259

Index of Figures

Figure 1:	Program context menu with menu entry for the plug-in	11
Figure 2:	Login dialog of the plug-in for the system login	12
Figure 3:	Initializing the plug-in	12
Figure 4:	Main window of the plug-in with variable selection tree	13
Figure 5:	Graphic visualization of process values as a function of time	14
Figure 6:	Button Start	14
Figure 7:	Button Pause	15
Figure 8:	Button Resume	15
Figure 9:	Button Stop	15
Figure 10:	Input fields Chart Time Length	15
Figure 11:	Button Save Picture	15
Figure 12:	Button Zoom to Rectangle	16
Figure 13:	Button Pan	16
Figure 14:	Button Home	16

MANUAL
SILworX Plug-In
O_Scope

For further information, please contact:

HIMA Paul Hildebrandt GmbH
Albert-Bassermann-Str. 28
68782 Brühl, Germany

Phone +49 6202 709-0
Fax +49 6202 709-107
E-mail info@hima.com

Learn more about HIMA solutions online:



www.hima.com/en/



www.hima.com